Learning Activity: Training a Neural Network by Hand — The XOR Problem

Backpropagation and Gradient Descent from Scratch
Fundamentals of AI with Neural Networks and Transformers

Student	Elián David Martínez Orozco
Professor	Dr. rer. nat. Humberto Llinás
Course	Fundamentals of AI (Neural Networks & Transformers)
Institution	Universidad del Norte, Barranquilla, Colombia
Date	April 2026
Reference	https://rpubs.com/hllinas/R_AI1_toc

Architecture: 2 inputs $\rightarrow$ 2 hidden (sigmoid) $\rightarrow$ 1 output (sigmoid) **Loss:** $E = \frac{1}{2}(y - \hat{y})^2$ **Learning rate:** $\alpha = 0.25$

No neural network libraries used — all backpropagation implemented from scratch with NumPy.

Contents

Abstract

This report documents the step-by-step training of a feedforward neural network applied to the XOR binary classification problem, following the class notes of Llinás (2026). Starting from the parameters at the end of Epoch 2, the third training epoch is computed manually and verified numerically, yielding $\hat{y} = 0.57924$ and $E = 0.08852$. Automated training over 15 epochs confirms monotone loss reduction from 0.09095 to 0.07520 under learning rate $\alpha = 0.25$. Experiments with an alternative initialization and a two-observation vectorized extension demonstrate that training data diversity is the primary driver of convergence speed. The full XOR dataset experiment (5,000 epochs, $\alpha = 1.0$) converges to a local minimum achieving only 50% accuracy, illustrating the non-convex nature of neural network optimization and the critical role of initialization. All computations are performed from scratch using NumPy, without any neural network or automatic differentiation library.

Introduction

The XOR (exclusive-OR) function is among the most instructive benchmarks in neural network theory. It is not linearly separable: no straight line in $\mathbb{R}^2$ can correctly partition the four input combinations.

Table 1: *The XOR truth table*

x_1	x_2	y	Interpretation
0	0	0	Same inputs → output 0
0	1	1	Different inputs → output 1
1	0	1	Different inputs → output 1
1	1	0	Same inputs → output 0

This activity continues from Chapter 12 of the class notes (Llinás, 2026) — *Example: Manual Forward Pass for a Single XOR Observation* — and extends the analysis through eight required sections.

Methodology

Network Architecture

The network reproduces Figure 12.1 of the class notes (Llinás, 2026): two input neurons, one hidden layer with two sigmoid neurons, and one output sigmoid neuron, with bias nodes b_1, b_2 (hidden) and b_3 (output), and six trainable weights $w_1, \ldots, w_6$.

Forward pass:

$$z_j = \sum_i w_{ij}\, x_i + b_j, \quad h_j = \sigma(z_j), \quad z_3 = w_5 h_1 + w_6 h_2 + b_3, \quad \hat{y} = \sigma(z_3)$$

Activation: $\sigma(z) = \dfrac{1}{1+e^{-z}}$ **Loss:** $E = \dfrac{1}{2}(y-\hat{y})^2$ **Update:** $w_i \leftarrow w_i - \alpha \dfrac{\partial E}{\partial w_i}$

Backpropagation equations:

$$\frac{\partial E}{\partial z_3} = (\hat{y} - y)\,\hat{y}(1-\hat{y}), \quad \frac{\partial E}{\partial z_1} = \frac{\partial E}{\partial z_3} \cdot w_5 \cdot h_1(1-h_1), \quad \frac{\partial E}{\partial z_2} = \frac{\partial E}{\partial z_3} \cdot w_6 \cdot h_2(1-h_2)$$

Initial parameters (Figure 12.2, Llinás 2026):

$$W^{(0,1)} = \begin{pmatrix} 0.1 & 0.5 \\ -0.7 & 0.3 \end{pmatrix}, \quad W^{(1,2)} = \begin{pmatrix} 0.2 \\ 0.4 \end{pmatrix}, \quad b_1 = b_2 = b_3 = 0$$

No functions from `keras`, `tensorflow`, `torch`, or `sklearn.neural_network` are used. All forward passes, backpropagation, and parameter updates are implemented from scratch.

§1 Manual Third Epoch (§1)

Starting from parameters at the end of Epoch 2: $w_3 = -0.69764$, $w_4 = 0.30517$, $w_5 = 0.21724$, $w_6 = 0.42985$; all biases and w_1, w_2 unchanged.

Forward Pass

$$z_1 = (0.1)(0) + (-0.69764)(1) + 0 = -0.69764, \quad h_1 = \sigma(-0.69764) = 0.33234$$
$$z_2 = (0.5)(0) + (0.30517)(1) + 0 = 0.30517, \quad h_2 = \sigma(0.30517) = 0.57571$$
$$z_3 = (0.21724)(0.33234) + (0.42985)(0.57571) = 0.31966, \quad \hat{y} = 0.57924$$
$$E = \tfrac{1}{2}(1 - 0.57924)^2 = \mathbf{0.08852}$$

Backpropagation

$$\frac{\partial E}{\partial z_3} = (0.57924 - 1)(0.57924)(0.42076) = -0.10255$$
$$\frac{\partial E}{\partial w_5} = -0.03408, \quad \frac{\partial E}{\partial w_6} = -0.05904, \quad \frac{\partial E}{\partial w_3} = -0.00494, \quad \frac{\partial E}{\partial w_4} = -0.01077$$
$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial w_2} = 0 \quad \text{(because } x_1 = 0\text{)}$$

Table 2: *Parameter evolution across three epochs*

	w_1	w_2	w_3	w_4	w_5	w_6	E	$\lvert err \rvert$
Epoch 1	0.10000	0.50000	−0.69884	0.30255	0.20865	0.41498	0.09095	0.42650
Epoch 2	0.10000	0.50000	−0.69765	0.30517	0.21724	0.42985	0.08973	0.42362
Epoch 3	0.10000	0.50000	−0.69640	0.30786	0.22576	0.44461	**0.08852**	0.42076

Loss decreases by approximately $\Delta E \approx -0.00121$ per epoch — nearly constant, characteristic of gradient descent on a locally smooth surface. The weights w_1, w_2 remain frozen because $x_1 = 0$ zeros their gradient identically: $\partial E/\partial w_1 = (\partial E/\partial z_1) \cdot x_1 = 0$.

§2 Automated Training for 15 Epochs (§2)

```
for ep in range(1, 16):
    z1   = W["w3"]*x2 + B["b1"]           # x1=0: frozen term
    h1, h2 = sigmoid(z1), sigmoid(z2)
    z3   = W["w5"]*h1 + W["w6"]*h2 + B["b3"]
    yhat = sigmoid(z3);  loss = 0.5*(y - yhat)**2
    dE_dz3 = (yhat - y)*yhat*(1 - yhat)    # chain rule
    W[k] -= alpha * dE_dw                  # gradient descent
```

Listing 1: *Core training loop — from scratch, no ML libraries*

Table 3: *Table 1 — Forward pass and loss metrics (selected epochs)*

Ep	z_1	z_2	z_3	h_1	h_2	$\hat{y}$	$\lvert err\rvert$	E	$\partial E/\partial z_3$
1	−0.70000	0.30000	0.29614	0.33181	0.57444	0.57350	0.42650	0.09095	−0.10432
3	−0.69765	0.30517	0.31967	0.33233	0.57571	0.57924	0.42076	0.08852	−0.10255
5	−0.69514	0.31062	0.34298	0.33289	0.57704	0.58491	0.41509	0.08615	−0.10078
8	−0.69110	0.31928	0.37756	0.33379	0.57915	0.59328	0.40672	0.08271	−0.09814
10	−0.68824	0.32535	0.40036	0.33442	0.58063	0.59877	0.40123	0.08049	−0.09639
12	−0.68525	0.33163	0.42296	0.33509	0.58216	0.60419	0.39581	0.07833	−0.09466
15	−0.68056	0.34142	0.45650	0.33614	0.58453	0.61218	0.38782	0.07520	−0.09207

Table 4: *Table 2 — Weight evolution (selected epochs)*

Ep	w_1	w_2	w_3	w_4	w_5	w_6
1	0.10000	0.50000	−0.69884	0.30255	0.20865	0.41498
5	0.10000	0.50000	−0.69383	0.31345	0.24260	0.47380
10	0.10000	0.50000	−0.68824	0.32535	0.27549	0.53085
15	0.10000	0.50000	−0.67894	0.34477	0.32288	0.61319

Frozen weights. w_1, w_2 never update because $x_1 = 0$ zeros their gradient on every epoch. **Slowing gradient.** $|\partial E/\partial z_3|$ decreases from 0.10432 to 0.09207 as $\hat{y} \to y$ — the sigmoid derivative $\hat{y}(1-\hat{y})$ shrinks simultaneously with the error term, producing the vanishing signal characteristic of sigmoid training.

§3 Graphical Analysis (§3)

The following descriptions summarize the four-panel training analysis (Figure 1 of the HTML report):

- **Panel (a) — Loss.** Smooth monotonically decreasing curve from $E_1 = 0.09095$ to $E_{15} = 0.07520$. Total reduction: 0.01575.
- **Panel (b) — Absolute error.** Mirrors the loss curve; $|e_1| = 0.42650$, $|e_{15}| = 0.38782$.
- **Panel (c) — Gradient magnitude.** $|\partial E/\partial z_3|$ decreases from 0.10432 to 0.09207; illustrates mild vanishing gradient.
- **Panel (d) — Weight evolution.** w_5, w_6 (output layer) update faster than w_3, w_4 (hidden layer); w_1, w_2 remain constant throughout.

Output-layer weights w_5, w_6 update faster because their gradient path avoids one additional chain-rule multiplication through the hidden activation derivative $h_j(1-h_j)$. This is the signature of the vanishing gradient through layers, visible even in this minimal two-layer architecture.

§ 4 Sensitivity to Initial Parameters (§4)

Alternative initialization: $w_1 = 0.6$, $w_2 = -0.4$, $w_3 = 0.8$, $w_4 = -0.3$, $w_5 = 0.5$, $w_6 = -0.6$, $b_1 = 0.1$, $b_2 = -0.1$, $b_3 = 0.05$.

Table 5: *Sensitivity comparison — 15 epochs*

Metric	Original init.	Alternative init.
Initial $\hat{y}$	0.57350	0.54108
Initial loss	0.09095	0.10530
Final $\hat{y}$ (Ep 15)	0.61218	0.67418
Final loss	0.07520	**0.05308**
Total loss reduction	0.01575	**0.05223**
Final $\lvert error \rvert$	0.38782	**0.32582**

Non-zero biases update every epoch regardless of input values: $\partial E/\partial b_j = \partial E/\partial z_j$. This provides three additional active gradient paths, explaining the $3.3\times$ improvement in total loss reduction. Neither initialization converges in 15 epochs — the structural frozen-weight limitation from $x_1 = 0$ persists.

§ 5 Two-Observation Vectorized Training (§5)

Training extends to two observations with a matrix-based forward and backward pass:

$$\mathbf{X} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \in \mathbb{R}^{2\times 2}, \quad \mathbf{y} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \in \mathbb{R}^{2\times 1}$$

$$\mathbf{Z}^{(1)} = \mathbf{X}\, W^{(0,1)} + \mathbf{1}\, b^{(1)\top}, \quad \mathbf{H}^{(1)} = \sigma(\mathbf{Z}^{(1)}), \quad \hat{\mathbf{Y}} = \sigma(\mathbf{H}^{(1)} W^{(1,2)} + b^{(2)})$$

Matrix backward pass:

$$\boldsymbol{\delta}^{(2)} = (\hat{\mathbf{Y}} - \mathbf{y}) \odot \hat{\mathbf{Y}} \odot (1 - \hat{\mathbf{Y}}), \quad \frac{\partial E}{\partial W^{(1,2)}} = \mathbf{H}^{(1)\top} \boldsymbol{\delta}^{(2)}, \quad \boldsymbol{\delta}^{(1)} = (\boldsymbol{\delta}^{(2)} W^{(1,2)\top}) \odot \mathbf{H}^{(1)} \odot (1 - \mathbf{H}^{(1)})$$

Table 6: *Two-observation training results (selected epochs)*

Ep	$\hat{y}_1$ (0,1)	$\hat{y}_2$ (1,0)	Mean Loss	Mean $\lvert err \rvert$	$\lVert \boldsymbol{\delta}^{(2)} \rVert$
1	0.57350	0.58758	0.08800	0.41946	0.14447
5	0.61986	0.63831	0.06883	0.37091	0.12246
10	0.66655	0.68863	0.05204	0.32241	0.09975
15	0.70317	0.72739	**0.04061**	0.28472	0.08222

The second observation $(1, 0) \mapsto 1$ provides $x_1 = 1$, activating gradient paths for w_1, w_2 for the first time. All six weights update, and the mean loss at Epoch 15 (0.04061) is nearly half the single-observation result (0.07520). Training data diversity — not learning rate or initialization — is the primary driver of convergence speed.

§ 6 Interpretation and Comparison (§6)

Table 7: *Three-experiment comparison*

Metric	1-obs original	1-obs alternative	2-obs
Initial $\hat{y}$	0.57350	0.54108	0.57350
Initial loss	0.09095	0.10530	0.08800
Final $\hat{y}$ (Ep 15)	0.61218	0.67418	**0.70317**
Final loss	0.07520	0.05308	**0.04061**
Total loss reduction	0.01575	0.05223	**0.04739**
Active weights	4 of 6	4 of 6 + biases	**6 of 6**

Why XOR requires a hidden layer. Without a hidden layer, $\hat{y} = \sigma(w_1 x_1 + w_2 x_2 + b)$ is a single sigmoid on a linear combination, producing only a linear decision boundary. The XOR pattern cannot be separated by any straight line in $\mathbb{R}^2$. The hidden layer generates nonlinear intermediate representations h_1, h_2 whose combination produces the required curved boundary.

§ 7 Conceptual Reflection (§7)

Each training iteration follows a five-stage learning cycle:

1. **Prediction $\hat{y}$.** Forward propagation computes $\hat{y} = 0.57350$ at Epoch 1 — determined entirely by initial weights, not the target.
2. **Error.** $y - \hat{y} = 1 - 0.57350 = 0.42650$ — the mistake to be corrected.
3. **Loss.** $E = \frac{1}{2}(y - \hat{y})^2$ encodes the error as a smooth, differentiable scalar bounded below by zero. Its derivative $\partial E / \partial \hat{y} = \hat{y} - y$ pushes $\hat{y}$ toward y.
4. **Gradient and backpropagation.** The chain rule propagates $\partial E / \partial z_3$ backward through $W^{(1,2)}$ and the hidden activation derivative, distributing proportional blame to each weight.
5. **Update.** $w_i \leftarrow w_i - \alpha \cdot \partial E / \partial w_i$. Every active weight moves in the direction reducing loss. After 15 epochs: $\hat{y}$ increases from 0.57350 to 0.61218; E decreases from 0.09095 to 0.07520.

The phrase *"learning from mistakes"* is mathematically precise: the loss encodes the mistake, the gradient distributes the blame proportionally, and the update applies a scaled correction. The improvement is incremental but guaranteed by the gradient descent convergence theorem on smooth, differentiable loss surfaces.

§ 8 Full XOR Table (§8)

The complete XOR dataset ($\mathbf{X} \in \mathbb{R}^{4\times2}$, $\mathbf{y} \in \mathbb{R}^{4\times1}$) is trained with $\alpha = 1.0$, $5{,}000$ epochs, and random initialization $\sim \mathcal{N}(0, 0.64)$ with `np.random.seed(42)`.

Final Predictions after 5,000 Epochs

Table 8: *Full XOR — final predictions and classification results*

x_1	x_2	y	$\hat{y}$ (prob.)	$\hat{y}$ (class)	Correct?
0	0	0	0.01726	0	✓
0	1	1	0.49948	0	×
1	0	1	0.98294	1	✓
1	1	0	0.50057	1	×
Accuracy					**2/4 = 50%**
Final mean loss					0.062709

Decision Boundary Analysis

The closed-form approximation consistent with all four predictions is:

$$\hat{y} \approx \sigma\big(8(x_1 - \tfrac{1}{2}) - 4x_2(2x_1 - 1)\big)$$

Verification: $(0,0) \to \sigma(-4) = 0.018$ ✓ · $(0,1) \to \sigma(0) = 0.500$ ✓ · $(1,0) \to \sigma(4) = 0.982$ ✓ · $(1,1) \to \sigma(0) = 0.500$ ✓

Boundary at $\hat{y} = 0.5$: Setting the argument to zero: $8(x_1 - \frac{1}{2}) - 4x_2(2x_1 - 1) = 0 \Rightarrow 4(2x_1 - 1)(1 - x_2) = 0 \Rightarrow x_1 = 0.5$ *or* $x_2 = 1$.

The decision boundary is the union of the vertical line $x_1 = 0.5$ and the horizontal line $x_2 = 1$. Points $(0,1)$ and $(1,1)$ lie exactly on this boundary ($\hat{y} \approx 0.5$), causing misclassification at the 0.5 threshold.

> The 50% accuracy result is not a code error. With `np.random.seed(42)`, gradient descent converges to a local minimum where the network learned to classify based primarily on whether $x_1 > 0.5$, rather than the true XOR function. The near-zero gradient at convergence confirms a critical point, but it is a saddle or local minimum — not the global solution. Practical remedies include multiple random restarts, momentum-based optimizers (Adam), or Xavier initialization.

Discussion

Gradient information requires non-zero inputs. The chain rule gives $\partial E/\partial w_1 = (\partial E/\partial z_1) \cdot x_1$. When $x_1 = 0$ identically, w_1 and w_2 are structurally frozen regardless of loss magnitude or learning rate. Two-observation training resolves this by providing $x_1 = 1$ through the second observation, activating all six gradient paths.

Vanishing gradient at small scale. Even in this 2-2-1 network, $|\partial E/\partial z_3|$ decreases monotonically. The product $(\hat{y} - y) \cdot \hat{y}(1 - \hat{y})$ shrinks as $\hat{y} \to y$, compounding across layers. In deep networks, this becomes exponential decay requiring ResNets, batch normalization, and ReLU activations.

Local minima in non-convex optimization. The full XOR experiment reaches 50% accuracy after 5,000 epochs despite a near-zero gradient at convergence. The network found a geometrically interpretable but incorrect separator: the union of lines $\{x_1 = 0.5\} \cup \{x_2 = 1\}$. Gradient descent provides no global optimality guarantee on non-convex loss surfaces.

Conclusions

1. **Manual verification confirmed.** Epoch 3 yields $\hat{y} = 0.57924$, $E = 0.08852$, $\Delta E \approx -0.00121$ — consistent with the class notes.
2. **Structural gradient freezing.** Training with $x_1 = 0$ permanently freezes w_1, w_2. Two-observation training resolves this and achieves $\approx 2\times$ lower final loss.
3. **Initialization matters.** Non-zero biases provide $3.3\times$ greater loss reduction in 15 epochs by activating additional gradient paths.
4. **Local minima prevent global convergence.** The full XOR experiment reaches a 50% accuracy local minimum, demonstrating that gradient descent provides no global optimality guarantee.
5. **A hidden layer is necessary.** No single sigmoid on a linear combination can represent XOR. At least one hidden layer is required to generate nonlinear intermediate representations.

Neural network training is governed simultaneously by *mathematical guarantees* (monotone loss reduction under appropriate α) and *practical limitations* (local minima, frozen gradients, sigmoid saturation). Both dimensions are visible in this minimal XOR example, making it an exceptionally instructive benchmark for understanding the theory and practice of gradient-based learning.

Reproducibility

Table 9: *Library versions*

#	Library	Version	Role
1	Python	3.12.13	Base language
2	numpy	2.0.2	Vector/matrix operations, sigmoid
3	pandas	2.2.2	DataFrames and result tables
4	matplotlib	3.10.0	Network diagram and training plots

Random seed: `np.random.seed(42)` **No ML/autodiff libraries.**

Llinás, H. (2026). *Foundations of AI: Neural Networks and Transformers.* RPubs. https://rpubs.com/hllinas/R_AI1_toc